

Lecture 21

Secure Computation: Oblivious Transfer, Garbled Circuits, and PSI

SEC 595: Encrypted Computing

Roadmap

Homomorphic encryption lets one party compute on another party's encrypted data. Secure multi-party computation (MPC) is the general setting: several parties, each with a private input, jointly compute a function of all inputs without trusting one another. Section 1 defines what “secure” means, through the real/ideal (simulation) paradigm. Section 2 gives the basic two-party primitive, oblivious transfer, with an RSA-based construction. Section 3 uses it in Yao's garbled circuits, which compute any function. Section 4 returns to homomorphic encryption for a specific task, private set intersection.

1. Defining Secure Computation

1.1 The Setting

K mutually distrustful parties hold private inputs $x_1, \dots, x_K$ and want to compute a functionality

$$F : (\{0, 1\}^*)^K \rightarrow (\{0, 1\}^*)^K,$$

where party i receives the i -th output. F may be deterministic or randomised, and is assumed computable in polynomial time. Typical examples: an election (output: whether the majority voted “yes”); an auction (output: the winning bid only); Yao's millionaires' problem, $F(A, B) = ([A \geq B], [A \leq B])$; and two companies comparing customer lists (output: the common customers).

All of these are trivial with a trusted third party: everyone sends their input to it, and it returns the outputs. The goal of MPC is to obtain the same result with no trusted party. The properties we want are:

- **Privacy:** each party learns its output and nothing more.
- **Correctness:** the outputs are F of the inputs.
- **Independence of inputs:** no party chooses its input as a function of another's.
- **Fairness:** if one party receives output, all do.

Listing properties is not a definition: the list may be incomplete. The simulation paradigm replaces it with a single comparison.

1.2 The Real/Ideal Paradigm

- In the **ideal model**, the parties send their inputs to a trusted party, which computes F and returns each party's output. This is secure by definition.

- In the **real model**, the parties run a protocol π by exchanging messages, with no trusted help.

A protocol is secure if whatever an adversary can achieve in the real model, it could also achieve in the ideal model. Since the ideal model leaks nothing beyond the outputs, neither can the protocol.

1.3 Adversary Models

Definition 1 (Semi-honest and malicious adversaries). A **semi-honest** (passive, honest-but-curious) adversary follows the protocol exactly but tries to learn more from the messages it receives. A **malicious** (active) adversary may deviate arbitrarily: send malformed messages, lie about its input, or abort at any point.

For the rest of the lecture we consider two parties, Alice and Bob, and semi-honest adversaries. For two parties, the *view* of Alice in an execution of π on inputs (x, y) is

$$\text{view}_A^\pi(x, y) = (x, \rho_A, \text{all messages Alice receives}),$$

where ρ_A is her random coins; view_B^π is defined in the same way. Everything Alice can learn is a function of her view.

Definition 2 (Security against semi-honest adversaries). Let $F = (F_A, F_B)$ be deterministic. A protocol π securely computes F in the semi-honest model if it is correct and there exist polynomial-time simulators $\mathcal{S}_A, \mathcal{S}_B$ such that, for all inputs x, y ,

$$\mathcal{S}_A(x, F_A(x, y)) \stackrel{c}{\approx} \text{view}_A^\pi(x, y), \quad \mathcal{S}_B(y, F_B(x, y)) \stackrel{c}{\approx} \text{view}_B^\pi(x, y),$$

where $\stackrel{c}{\approx}$ denotes computational indistinguishability.

The simulator receives only what the party is entitled to in the ideal model — its own input and output — and must produce something that looks like the real view. If it can, the view contains nothing more.

For malicious adversaries the ideal model is enlarged: the corrupted party may substitute any input of its choice and may abort. Security then requires that for every real adversary there is an ideal adversary such that the outputs of the two executions, *together with the honest party's output*, are indistinguishable. Including the honest output is what captures correctness and independence of inputs.

1.4 What a Proof Looks Like

A semi-honest security proof has the same shape every time.

1. Write down each party's view: its input, coins and received messages.
2. Build a simulator that, from the party's input and output alone, samples a fake view.
3. Show that the fake and real views are identically distributed, or indistinguishable by a reduction to a hardness assumption (RSA, IND-CPA of an encryption scheme, and so on).

Section 2.3 carries this out for oblivious transfer.

2. Oblivious Transfer

2.1 Definition

Definition 3 (1-out-of-2 oblivious transfer). The sender holds two messages m_0, m_1 ; the receiver holds a bit $b \in \{0, 1\}$. The functionality is

$$F_{\text{OT}}((m_0, m_1), b) = (\perp, m_b) :$$

the sender learns nothing, and the receiver learns m_b and nothing about m_{1-b} .

The two requirements pull in opposite directions: the sender must not learn which message was taken, yet must be sure only one was. OT is a fundamental primitive: any function can be computed securely from OT alone. The generalisation *1-out-of- n OT*, where the receiver picks one of n messages, can be built from 1-out-of-2 OTs (Exercise 3).

2.2 An RSA-Based Protocol

Alice is the sender, Bob the receiver. Arithmetic is modulo the RSA modulus n (Lecture 1), and messages lie in $\mathbb{Z}_n$.

Definition 4 (RSA-based 1-out-of-2 OT). 1. Alice generates an RSA key pair $\text{Pk} = (n, e)$, $\text{Sk} = d$, and two random values $r_0, r_1 \in \mathbb{Z}_n$. She sends (n, e, r_0, r_1) to Bob.
2. Bob, with choice bit b , picks a random $k \in \mathbb{Z}_n$ and sends

$$v = r_b + k^e \bmod n.$$

3. Alice computes both candidate keys

$$k_0 = (v - r_0)^d \bmod n, \quad k_1 = (v - r_1)^d \bmod n,$$

and sends $m'_0 = m_0 + k_0$ and $m'_1 = m_1 + k_1 \pmod n$.

4. Bob outputs $m_b = m'_b - k \bmod n$.

Correctness. $k_b = (v - r_b)^d = (k^e)^d = k$, so $m'_b - k = m_b$. The other key is $k_{1-b} = (k^e + r_b - r_{1-b})^d$, an unrelated value.

Example 1 (OT with $n = 143$). Use the RSA keys of Lecture 8: $n = 143$, $e = 7$, $d = 103$. Alice's messages are $m_0 = 42$, $m_1 = 99$; Bob's choice is $b = 1$.

1. Alice sends $r_0 = 20$, $r_1 = 55$.
2. Bob picks $k = 5$: $k^e = 5^7 \bmod 143 = 47$, so $v = 55 + 47 = 102$.
3. Alice computes $k_0 = (102 - 20)^{103} = 82^{103} \bmod 143 = 4$ and $k_1 = (102 - 55)^{103} = 47^{103} \bmod 143 = 5$. She sends $m'_0 = 42 + 4 = 46$ and $m'_1 = 99 + 5 = 104$.
4. Bob computes $m'_1 - k = 104 - 5 = 99 = m_1$. ✓ Subtracting k from the other message gives $46 - 5 = 41$, which is not m_0 .

2.3 Security Against Semi-Honest Parties

Proposition 1. *The RSA-based OT is secure against a semi-honest sender, perfectly; and against a semi-honest receiver, under the RSA assumption (proof sketch).*

Proof sketch. Alice's view is $(m_0, m_1, d, r_0, r_1, v)$. Because $k \mapsto k^e$ is a permutation of $\mathbb{Z}_n$, k^e is uniform when k is, so $v = r_b + k^e$ is uniform on $\mathbb{Z}_n$ whatever b is. The simulator $\mathcal{S}_A$ outputs Alice's input, fresh keys and coins, and a uniform v ; the distributions are identical.

Bob's view is $(b, k, n, e, r_0, r_1, m'_0, m'_1)$, and his output is m_b . $\mathcal{S}_B$ picks the key, r_0, r_1 and k itself, sets $m'_b = m_b + k$, and chooses m'_{1-b} uniformly at random. The only difference from the real view is $m'_{1-b} = m_{1-b} + k_{1-b}$, where k_{1-b} is the e -th root of the value $v - r_{1-b}$. Computing it without d is inverting RSA, which is assumed hard, so k_{1-b} masks m_{1-b} . $\square$

Remark. Hardness of computing k_{1-b} does not by itself make $m_{1-b} + k_{1-b}$ indistinguishable from random; a rigorous version masks m_{1-b} with a value derived from k_{1-b} (for instance a hash). Note also that the protocol is only semi-honest secure: nothing stops a malicious party from sending values other than those prescribed.

3. Yao's Garbled Circuits

3.1 Idea

Any efficiently computable function can be written as a polynomial-size Boolean circuit of AND, OR, XOR and NOT gates. Yao's protocol lets Alice and Bob evaluate such a circuit for $F(X, Y)$ on their private inputs.

- Alice, the **garbler** (constructor), encrypts the circuit.
- Bob, the **evaluator**, evaluates the encrypted circuit without learning any intermediate value.

3.2 Garbling a Gate

Definition 5 (Wire keys and garbled table). For every wire w Alice picks two random keys k_w^0, k_w^1 , standing for the values 0 and 1. For a gate g with input wires x, y and output wire z , the **garbled table** consists of the four ciphertexts

$$E_{k_x^a} \left(E_{k_y^b} (k_z^{g(a,b)}) \right), \quad a, b \in \{0, 1\},$$

listed in a random order. E is a symmetric encryption scheme.

Each row encrypts the output key for one input combination under the two input keys for that combination (double encryption). The rows are randomly permuted, so their position reveals nothing. Alice keeps the mapping between keys and bits to herself.

Example 2 (An AND gate). For $z = x \wedge y$ the truth table and its encryption are

x	y	z	row
0	0	0	$E_{k_x^0}(E_{k_y^0}(k_z^0))$
0	1	0	$E_{k_x^0}(E_{k_y^1}(k_z^0))$
1	0	0	$E_{k_x^1}(E_{k_y^0}(k_z^0))$
1	1	1	$E_{k_x^1}(E_{k_y^1}(k_z^1))$

Alice sends the four rows shuffled, for instance in the order $(0, 1), (1, 1), (1, 0), (0, 0)$. If Bob holds k_x^0 and k_y^1 , exactly one row — $E_{k_x^0}(E_{k_y^1}(k_z^0))$ — decrypts, and he obtains k_z^0 . He does not learn that $x = 0, y = 1$ or $z = 0$: he holds three random strings.

Recognising the right row. Bob does not know which row matches his keys, so he tries all four. To make the correct one recognisable, Alice appends a string of zeros to each output key before encrypting, $k_z^c \parallel 00000$. With the wrong keys, decryption yields a random string, which ends in the zero pattern only with negligible probability.

Evaluation. Bob processes the gates in topological order. At each gate he holds one key per input wire, decrypts the one matching row, and obtains one key for the output wire. Every wire carries exactly one key, which corresponds to 0 or 1, and Bob does not know which.

3.3 Yao's Two-Party Protocol

Bob needs the keys for his own input bits, but Alice must not learn those bits, and Bob must not receive both keys of a wire (with both, he could evaluate the circuit on other inputs). This is exactly oblivious transfer.

- Definition 6** (Yao's protocol). 1. Alice garbles every gate of the circuit for F .
2. She sends Bob the garbled tables and the keys for her own input bits. Since keys are random, these reveal nothing about her bits.
3. For each of Bob's input wires w , Alice and Bob run a 1-out-of-2 OT: Alice's messages are (k_w^0, k_w^1) and Bob's choice is his bit on w . Bob receives k_w^{bit} ; Alice learns nothing.
4. Bob evaluates the garbled circuit and obtains one key per output wire.
5. Output decoding: either Alice sends Bob the decoding table $\{k_o^0 \mapsto 0, k_o^1 \mapsto 1\}$ for each output wire o , and Bob reports the result; or Bob sends the output keys to Alice, who decodes them.

Intermediate keys are never revealed: decoding them would show intermediate values. The protocol uses a constant number of rounds, four ciphertexts per gate, and one OT per bit of Bob's input. It is secure against semi-honest parties: Alice's view consists only of her side of the OTs, and Bob's view of random keys, one per wire, together with the output.

Example 3 (A full adder). The full adder adds bits A , B and a carry-in C_{in} :

$$D = A \oplus B, \quad S = D \oplus C_{in}, \quad E = D \wedge C_{in}, \quad F = A \wedge B, \quad C_{out} = E \vee F,$$

computed by gates $G_1, \dots, G_5$ in that order. Alice holds $A = 1$ and $C_{in} = 0$; Bob holds $B = 0$.

Garbling. Alice picks keys k_w^0, k_w^1 for each of the eight wires $A, B, C_{in}, D, E, F, S, C_{out}$ and builds five garbled tables of four rows each.

Inputs. Alice sends the tables and her keys $k_A^1, k_{C_{in}}^0$. An OT with Bob's bit $B = 0$ gives Bob k_B^0 .

Evaluation. In each gate exactly one row decrypts:

Gate	Bob's keys	Row that decrypts
G_1 (XOR)	k_A^1, k_B^0	$E_{k_A^1}(E_{k_B^0}(k_D^1)) \rightarrow k_D^1$
G_2 (XOR)	$k_D^1, k_{C_{in}}^0$	$E_{k_D^1}(E_{k_{C_{in}}^0}(k_S^1)) \rightarrow k_S^1$
G_3 (AND)	$k_D^1, k_{C_{in}}^0$	$E_{k_D^1}(E_{k_{C_{in}}^0}(k_E^0)) \rightarrow k_E^0$
G_4 (AND)	k_A^1, k_B^0	$E_{k_A^1}(E_{k_B^0}(k_F^0)) \rightarrow k_F^0$
G_5 (OR)	k_E^0, k_F^0	$E_{k_E^0}(E_{k_F^0}(k_{C_{out}}^0)) \rightarrow k_{C_{out}}^0$

Output. Bob holds $\{k_S^1, k_{C_{out}}^0\}$, which decode to $S = 1, C_{out} = 0$: indeed $1 + 0 + 0 = (01)_2$. ✓ Bob never learns D, E or F .

4. Private Set Intersection

4.1 The Problem

Definition 7 (Private set intersection). Alice holds a set $X = \{x_1, \dots, x_k\}$ and Bob holds a set $Y = \{y_1, \dots, y_k\}$. The PSI functionality gives Alice $X \cap Y$ and Bob nothing.

Garbled circuits could compute this, but a direct protocol using additively homomorphic encryption is simpler. It combines two earlier ideas: encoding a set as the roots of a polynomial (Lecture 2), and evaluating a polynomial with encrypted coefficients using Paillier (Lecture 8).

4.2 The Protocol

Let (Enc, Dec) be Paillier with modulus n , so that $\text{Enc}(a) \cdot \text{Enc}(b) = \text{Enc}(a + b)$ and $\text{Enc}(a)^c = \text{Enc}(ca)$, all modulo n . Set elements are in $\mathbb{Z}_n$.

Definition 8 (PSI by root encoding). 1. Alice forms the polynomial whose roots are her elements,

$$P(x) = (x_1 - x)(x_2 - x) \cdots (x_k - x) = a_0 + a_1x + \cdots + a_kx^k,$$

and sends Pk and $\text{Enc}(a_0), \dots, \text{Enc}(a_k)$ to Bob.

2. For each $y \in Y$, Bob evaluates the polynomial homomorphically,

$$\text{Enc}(P(y)) = \text{Enc}(a_0) \cdot \text{Enc}(a_1)^y \cdot \text{Enc}(a_2)^{y^2} \cdots \text{Enc}(a_k)^{y^k},$$

picks a fresh random r , and computes

$$\text{Enc}(rP(y) + y) = \text{Enc}(P(y))^r \cdot \text{Enc}(y).$$

3. Bob sends the k results to Alice in a random order.

4. Alice decrypts each result; the values that lie in X form $X \cap Y$.

Proposition 2. If $y \in X$, the result decrypts to y . If $y \notin X$, it decrypts to a value that is uniform on $\mathbb{Z}_n$ whenever $P(y)$ is invertible modulo n .

Proof. If $y \in X$ then $P(y) = 0$, so $rP(y) + y = y$. If $y \notin X$ and $P(y)$ is a unit, then $r \mapsto rP(y) + y$ is a bijection of $\mathbb{Z}_n$, so a uniform r gives a uniform result. $\square$

A uniform value falls in X with probability only k/n , which is negligible.

Security sketch. Bob sees only Paillier ciphertexts, so his view is simulated by encryptions of 0, by IND-CPA security (Lecture 8, §1.5). Alice's view is a shuffled list containing the elements of $X \cap Y$ and uniformly random values; a simulator given $X \cap Y$ (and k) produces exactly this list. Note that Alice learns $|Y|$, which is therefore part of the output.

Cost. Communication is $O(k)$: $k + 1$ coefficients one way and k results back. Alice performs $O(k)$ encryptions and decryptions; Bob performs $k + 1$ exponentiations per element, $O(k^2)$ in total.

Example 4 (PSI with $n = 77$). Alice has $X = \{3, 4, 5\}$, Bob has $Y = \{5, 6\}$. Use the Paillier keys of Lecture 8: $n = 77$, $n^2 = 5929$, $g = 5$, $\lambda = 30$, $\mu = 37$.

Polynomial. $(3 - x)(4 - x) = 12 - 7x + x^2$, and multiplying by $(5 - x)$,

$$P(x) = 60 - 47x + 12x^2 - x^3.$$

Modulo 77 the coefficients are $(60, 30, 12, 76)$. Alice encrypts them with randomisers 2, 3, 4, 6, sending

$$\text{Enc}(60) = 1943, \quad \text{Enc}(30) = 5234, \quad \text{Enc}(12) = 3040, \quad \text{Enc}(76) = 3120.$$

Bob, $y = 5$. In plaintext, $P(5) = 60 - 235 + 300 - 125 = 0$. Homomorphically, with exponents 1, 5, 25, 125,

$$1943 \cdot 5234^5 \cdot 3040^{25} \cdot 3120^{125} \equiv 1943 \cdot 3818 \cdot 3642 \cdot 4168 \equiv 4195 \pmod{5929},$$

which decrypts to 0. Bob picks $r = 3$ and encrypts $y = 5$ with randomiser 8, getting 2488; the result is $4195^3 \cdot 2488 \equiv 4607 \cdot 2488 \equiv 1459$.

Bob, $y = 6$. In plaintext, $P(6) = 60 - 282 + 432 - 216 = -6 \equiv 71$. Homomorphically, with exponents 1, 6, 36, 216,

$$1943 \cdot 5234^6 \cdot 3040^{36} \cdot 3120^{216} \equiv 1943 \cdot 2682 \cdot 5328 \cdot 4173 \equiv 2790,$$

which decrypts to 71. Bob picks $r = 10$ and encrypts 6 with randomiser 9, getting 1901; the result is $2790^{10} \cdot 1901 \equiv 3532 \cdot 1901 \equiv 2704$.

Alice decrypts $1459 \mapsto 5 \in X$ and $2704 \mapsto 23 = 10 \cdot (-6) + 6 \pmod{77}$, which is not in X . She outputs $X \cap Y = \{5\}$. ✓ She learns nothing about 6.

Summary

1. Secure computation asks parties to compute F of private inputs as if a trusted party did it. A protocol is secure if a simulator, given only a party's input and output, can produce an indistinguishable view.
2. Semi-honest adversaries follow the protocol and try to learn more; malicious adversaries deviate arbitrarily. Malicious security also compares the honest party's output.
3. In 1-out-of-2 OT the receiver obtains one of two messages and the sender learns nothing. In the RSA construction $v = r_b + k^e$ is uniform, and only k_b equals k .
4. Yao's garbled circuits encrypt each gate as four doubly encrypted, shuffled rows. The evaluator learns one random key per wire; his input keys come from OT.
5. PSI encodes Alice's set as the roots of P . Bob evaluates P under Paillier and returns $\text{Enc}(rP(y) + y)$, which decrypts to y exactly when $y \in X$.

Exercises

1. Repeat Example 1 with $b = 0$ (same r_0, r_1, k). Compute v, k_0, k_1, m'_0, m'_1 , and verify that Bob recovers 42.
2. In the RSA-based OT, suppose a semi-honest Bob tried to obtain both messages. What value would he have to compute, and why does the RSA assumption prevent it?
3. Build a 1-out-of-4 OT from 1-out-of-2 OTs. (Hint: let the receiver's index be two bits; use random keys per bit to mask each message.)
4. Write the garbled table of an OR gate with keys $k_x^0, k_x^1, k_y^0, k_y^1, k_z^0, k_z^1$. Which row does Bob decrypt when $x = 0, y = 1$, and what does he learn?

5. Trace Example 3 with $A = 1$, $B = 1$, $C_{in} = 1$. List the key Bob obtains on each wire and the decoded output.
6. Run the PSI protocol in plaintext over $\mathbb{Z}_{77}$ for $X = \{2, 7\}$ and $Y = \{7, 9\}$, with masks $r = 4$ for $y = 7$ and $r = 5$ for $y = 9$. What values does Alice decrypt?
7. In the PSI protocol, why must Bob use a fresh r for each y , and why does he shuffle the results? What would Alice learn otherwise?